

Simulados Finais - Prova 4

Tecnologia e Programação para Dispositivos Móveis

Dart, orientação a objetos, cálculo aplicado, Firebase Functions e Firestore

Nome:		RA:	
Disciplina:	TPDM - Prova 4	Data:	___/___/___
Observação:	Use os textos de contextualização como fonte das regras e fórmulas.		

Material de estudo em formato de prova. Cada simulado possui um texto de contextualização e quatro questões diretas. As fórmulas e regras necessárias ficam no texto inicial, portanto releia a contextualização antes de implementar.

Estrutura do arquivo

- Simulado 1: recomendação inteligente por similaridade vetorial.
- Simulado 2: detecção de anomalias em sensores de um campus inteligente.
- Ao final, há um gabarito comentado com possíveis soluções.

SIMULADO 1

Tema: recomendação inteligente de trilhas de estudo

Simulado 1 - Contextualização

Em muitos sistemas modernos, uma recomendação não é feita apenas comparando textos de forma literal. Plataformas de estudo, lojas, redes sociais e mecanismos de busca costumam transformar características em números. Quando isso acontece, cada item passa a ser representado por um vetor. Um vetor pode ser entendido, neste simulado, como uma lista ordenada de valores numéricos. Cada posição dessa lista representa uma característica fixa. Por exemplo, em uma plataforma de estudos, a posição 0 pode representar interesse em lógica, a posição 1 pode representar orientação a objetos, a posição 2 pode representar Firebase, a posição 3 pode representar Flutter e a posição 4 pode representar inteligência artificial. Assim, o vetor [0.9, 0.7, 0.2, 0.4, 0.8] indica um perfil com alto interesse em lógica e IA, interesse intermediário em orientação a objetos e Flutter, e baixo interesse em Firebase.

A comparação entre dois vetores deve ser feita sempre respeitando a posição de cada característica. O valor da posição 0 de um vetor deve ser comparado com o valor da posição 0 do outro vetor, e assim por diante. Por esse motivo, dois vetores só podem ser comparados corretamente se tiverem o mesmo tamanho. Quando os tamanhos são diferentes, há um erro lógico: não existe correspondência completa entre as características. Em código, esse caso deve ser tratado como entrada inválida, normalmente lançando uma exceção como `ArgumentError`.

Uma das medidas mais usadas para comparar vetores em sistemas de recomendação é a similaridade do cosseno. A ideia principal é comparar a direção dos vetores, e não apenas a distância numérica direta entre eles. Dois vetores que apontam em direções parecidas são considerados semelhantes. Para calcular essa medida, usamos três partes: produto escalar, módulo do primeiro vetor e módulo do segundo vetor. O produto escalar é calculado multiplicando os valores correspondentes de cada posição e somando os resultados. Para os vetores A e B, $\text{produtoEscalar} = A[0] * B[0] + A[1] * B[1] + \dots + A[n] * B[n]$. O módulo de um vetor é calculado pela raiz quadrada da soma dos quadrados de seus valores: $\text{modulo}(A) = \text{raiz}(A[0]^2 + A[1]^2 + \dots + A[n]^2)$. Por fim, $\text{similaridade} = \text{produtoEscalar} / (\text{modulo}(A) * \text{modulo}(B))$.

Valores próximos de 1.0 indicam que os vetores possuem direções muito parecidas. Valores próximos de 0.0 indicam pouca relação entre os vetores. Valores negativos indicam direções opostas, embora em muitos exercícios com notas de interesse entre 0 e 1 esse caso seja menos comum. Se algum vetor tiver módulo igual a zero, significa que ele não possui força em nenhuma característica. Nesse simulado, a similaridade com um vetor de módulo zero deve ser tratada como 0.0, evitando divisão por zero.

Além da similaridade, um sistema real pode considerar outros critérios. Um conteúdo de estudo pode ter um campo ativo, indicando se ele pode aparecer nas recomendações; uma dificuldade, variando de 1 a 5; e uma prioridade, que aumenta ou reduz sua relevância no ranking. Neste simulado, o score final de um conteúdo será calculado como $\text{similaridade} * \text{prioridade}$. Um conteúdo inativo não deve ser recomendado. Um conteúdo com dificuldade maior que a dificuldade máxima informada pelo usuário também deve ser descartado. Após calcular o score final, os conteúdos restantes devem ser ordenados do maior score para o menor. Em caso de empate, o conteúdo de menor dificuldade deve aparecer primeiro.

Também é comum registrar o histórico de recomendações. Em um app Flutter integrado ao Firebase, a recomendação pode ser calculada em uma Cloud Function. Quando a função é `onCall`, ela é chamada diretamente pelo app usando o SDK do Firebase. Esse tipo de função recebe os dados em `request.data` e também pode acessar `request.auth` para verificar se o usuário está autenticado. Quando ocorre um erro de validação ou autenticação em uma função `onCall`, a resposta adequada é lançar `HttpsError`. Para acessar o Firestore no backend, usa-se o Firebase Admin SDK, inicializando o app com `initializeApp()` e obtendo o banco com `getFirestore()`.

Atenção: nas questões, não repita a explicação das fórmulas. Use as regras apresentadas nesta contextualização.

Simulado 1 - Questões

Resolva as questões em Dart, exceto a Questão 4, que deve ser resolvida em TypeScript usando Firebase Functions.

Questão 1. Implemente, em Dart, a função `calcularSimilaridadeCosseno`. Ela deve receber dois vetores de `double` e retornar a similaridade entre eles seguindo as regras do texto.

Resposta:

Questão 2. Crie a classe `ConteudoEstudo` e implemente a função `recomendarConteudos`. A função deve receber o perfil do usuário, uma lista de conteúdos e a dificuldade máxima permitida, retornando os três melhores conteúdos.

Questão 4. Implemente, em TypeScript, uma Firebase Function onCall chamada registrarRecomendacao. Ela deve validar autenticação, ler conteúdos ativos no Firestore, calcular o ranking, salvar o histórico da recomendação e retornar os três melhores itens.

Resposta:

SIMULADO 2

Tema: detecção de anomalias em sensores de campus inteligente

Simulado 2 - Contextualização

Um campus inteligente pode possuir sensores espalhados por salas, laboratórios, corredores e áreas abertas. Esses sensores podem medir temperatura, ruído, luminosidade, presença ou consumo de energia. Cada leitura de sensor pode ser registrada com um identificador do sensor, um tipo, um valor numérico e um instante de coleta. O objetivo de um sistema de monitoramento não é apenas armazenar leituras, mas também identificar comportamentos fora do esperado. Uma temperatura muito alta em um laboratório, um ruído muito elevado em uma biblioteca ou uma queda brusca de luminosidade em uma área externa podem indicar situações que exigem atenção.

Para analisar leituras, uma técnica simples é a média móvel. A média móvel usa uma janela de valores recentes. Se a janela for 3, cada posição da saída deve considerar o valor atual e até dois valores anteriores. Por exemplo, para a lista [10, 20, 30, 40] com janela 3, a primeira média usa apenas [10], a segunda usa [10, 20], a terceira usa [10, 20, 30] e a quarta usa [20, 30, 40]. A média móvel suaviza variações bruscas e ajuda a enxergar tendências. A janela deve ser maior que zero; caso contrário, a entrada é inválida.

Outra técnica útil é o z-score. O z-score mede o quanto um valor está distante da média em termos de desvios padrão. Primeiro calcula-se a média dos valores: $media = \text{soma dos valores} / \text{quantidade}$. Depois calcula-se o desvio padrão populacional: $desvio = \sqrt{\text{soma}((\text{valor} - \text{media})^2) / \text{quantidade}}$. Por fim, para uma leitura específica, $z = (\text{valor} - \text{media}) / \text{desvio}$. Se o desvio padrão for zero, todos os valores são iguais ou não há variação suficiente; nesse caso, para este simulado, o z-score deve ser considerado 0.0.

O valor absoluto do z-score indica o grau de afastamento da leitura em relação ao padrão. Quanto maior o valor absoluto, mais incomum é a leitura. Neste simulado, uma leitura será considerada anômala quando $\text{abs}(zScore) \geq \text{limiteZ}$. Um limite comum para exercícios é 2.0, mas a função deve receber esse limite por parâmetro. A severidade do alerta será classificada assim: se $\text{abs}(zScore)$ for menor que 2.0, não há alerta; se for maior ou igual a 2.0 e menor que 3.0, a severidade é moderada; se for maior ou igual a 3.0, a severidade é crítica.

Como as leituras podem vir de sensores diferentes, o sistema deve preservar a informação de tipo e identificador. Um alerta deve guardar a leitura que gerou o problema, o z-score calculado e a severidade. Também é útil gerar resumos por tipo de sensor. Um resumo pode contar quantos alertas ocorreram para cada tipo, qual foi o maior z-score absoluto observado e quantos alertas críticos existem. Esse tipo de resultado pode ser representado por Map em Dart.

Em um backend com Firebase Functions, uma função `onRequest` se comporta como uma rota HTTP. Ela recebe `req` e `res`, valida o método HTTP, lê dados do body ou da query string, usa códigos HTTP adequados e envia JSON como resposta. Em um cenário de sensores, um app ou dispositivo poderia enviar uma leitura por POST. A função salvaria a leitura no Firestore, consultaria leituras recentes do mesmo sensor, calcularia o z-score da nova leitura e, se necessário, salvaria um documento na coleção `alertas`. Essa abordagem centraliza a regra de detecção no servidor, evitando que cada cliente implemente a lógica de forma diferente.

Atenção: nas questões, use a média móvel, o z-score e a regra de severidade exatamente como definidos nesta contextualização.

Simulado 2 - Questões

Resolva as questões em Dart, exceto a Questão 4, que deve ser resolvida em TypeScript usando Firebase Functions.

Questão 1. Implemente, em Dart, a função `calcularMediaMovel`. Ela deve receber uma lista de `double` e o tamanho da janela, retornando a lista de médias conforme as regras do texto.

Resposta:

Questão 2. Crie as classes `LeituraSensor` e `AlertaSensor`. Depois implemente `detectarAnomalias`, recebendo uma lista de leituras de um mesmo sensor e o `limiteZ`.

Resposta:

Questão 4. Implemente, em TypeScript, uma Firebase Function onRequest chamada receberLeituraSensor. Ela deve aceitar apenas POST, validar os dados, salvar a leitura, calcular o z-score com base nas leituras recentes do mesmo sensor, registrar alerta quando necessário e retornar JSON.

Resposta:

GABARITO COMENTADO

Possíveis resoluções e explicações

Gabarito - Simulado 1

As soluções abaixo são uma possibilidade de resolução. Em prova, pequenas diferenças de nomes, organização e estrutura podem estar corretas se a lógica, as validações e o resultado estiverem adequados.

Questão 1 - calcularSimilaridadeCosseno

```
import 'dart:math';

double calcularSimilaridadeCosseno(List<double> vetorA, List<double> vetorB) {
  if (vetorA.length != vetorB.length) {
    throw ArgumentError('Os vetores precisam ter o mesmo tamanho.');
```

```
  }

  if (vetorA.isEmpty) {
    throw ArgumentError('Os vetores não podem estar vazios.');
```

```
  }

  double produtoEscalar = 0.0;
  double somaQuadradosA = 0.0;
  double somaQuadradosB = 0.0;

  for (int i = 0; i < vetorA.length; i++) {
    produtoEscalar += vetorA[i] * vetorB[i];
    somaQuadradosA += vetorA[i] * vetorA[i];
    somaQuadradosB += vetorB[i] * vetorB[i];
  }

  final double moduloA = sqrt(somaQuadradosA);
  final double moduloB = sqrt(somaQuadradosB);

  if (moduloA == 0 || moduloB == 0) {
    return 0.0;
  }

  return produtoEscalar / (moduloA * moduloB);
}

void main() {
  final perfil = [0.9, 0.7, 0.2, 0.4, 0.8];
  final conteudo = [1.0, 0.8, 0.1, 0.3, 0.9];

  print(calcularSimilaridadeCosseno(perfil, conteudo));
}
```

Comentário: A solução valida primeiro se os vetores podem ser comparados. Depois percorre as duas listas uma única vez, acumulando produto escalar e as somas necessárias para os módulos. O retorno 0.0 no caso de módulo zero evita divisão por zero, exatamente como o texto determinou.

Questão 2 - ConteudoEstudo e recomendarConteudos

```
class ConteudoEstudo {
  final String id;
  final String titulo;
  final List<double> vetor;
  final int dificuldade;
  final bool ativo;
  final double prioridade;
  double scoreFinal;

  ConteudoEstudo({
    required this.id,
    required this.titulo,
    required this.vetor,
    required this.dificuldade,
    required this.ativo,
    required this.prioridade,
    this.scoreFinal = 0.0,
  });
}

List<ConteudoEstudo> recomendarConteudos({
  required List<double> perfilUsuario,
  required List<ConteudoEstudo> conteudos,
  required int dificuldadeMaxima,
}) {
  final List<ConteudoEstudo> candidatos = [];

  for (final conteudo in conteudos) {
    if (!conteudo.ativo) {
      continue;
    }
  }
}
```

```

    }

    if (conteudo.dificuldade > dificuldadeMaxima) {
        continue;
    }

    final similaridade = calcularSimilaridadeCosseno(
        perfilUsuario,
        conteudo.vetor,
    );

    conteudo.scoreFinal = similaridade * conteudo.prioridade;
    candidatos.add(conteudo);
}

candidatos.sort((a, b) {
    final comparacaoScore = b.scoreFinal.compareTo(a.scoreFinal);

    if (comparacaoScore != 0) {
        return comparacaoScore;
    }

    return a.dificuldade.compareTo(b.dificuldade);
});

if (candidatos.length <= 3) {
    return candidatos;
}

return candidatos.sublist(0, 3);
}

void main() {
    final perfil = [0.9, 0.7, 0.2, 0.4, 0.8];

    final conteudos = [
        ConteudoEstudo(
            id: 'c1',
            titulo: 'Vetores em IA',
            vetor: [1.0, 0.6, 0.1, 0.2, 1.0],
            dificuldade: 3,
            ativo: true,
            prioridade: 1.2,
        ),
        ConteudoEstudo(
            id: 'c2',
            titulo: 'Firebase Functions',
            vetor: [0.2, 0.4, 1.0, 0.8, 0.3],
            dificuldade: 4,
            ativo: true,
            prioridade: 1.0,
        ),
        ConteudoEstudo(
            id: 'c3',
            titulo: 'Introdução a OOP',
            vetor: [0.7, 1.0, 0.1, 0.3, 0.2],
            dificuldade: 2,
            ativo: true,
            prioridade: 1.1,
        ),
        ConteudoEstudo(
            id: 'c4',
            titulo: 'Conteúdo inativo',
            vetor: [1.0, 1.0, 1.0, 1.0, 1.0],
            dificuldade: 1,
            ativo: false,
            prioridade: 5.0,
        ),
    ];

    final recomendados = recomendarConteudos(
        perfilUsuario: perfil,
        conteudos: conteudos,
        dificuldadeMaxima: 3,
    );

    for (final item in recomendados) {
        print('${item.titulo} - score: ${item.scoreFinal.toStringAsFixed(3)}');
    }
}

```

Comentário: A classe concentra os dados do conteúdo. A função filtra itens inativos e difíceis demais, calcula o score final usando a função da questão anterior e ordena a lista. O sort coloca maior score primeiro; se houver empate, a menor dificuldade ganha prioridade.

Questão 3 - gerarResumoRecomendacao

```
Map<String, dynamic> gerarResumoRecomendacao(List<ConteudoEstudo> recomendados) {
    if (recomendados.isEmpty()) {
        return {
            'totalRecomendado': 0,
            'mediaDificuldade': 0.0,
            'maiorScore': 0.0,
            'titulos': <String>[],
        };
    }

    int somaDificuldades = 0;
    double maiorScore = recomendados.first.scoreFinal;
    final List<String> titulos = [];

    for (final conteudo in recomendados) {
        somaDificuldades += conteudo.dificuldade;
        titulos.add(conteudo.titulo);

        if (conteudo.scoreFinal > maiorScore) {
            maiorScore = conteudo.scoreFinal;
        }
    }

    return {
        'totalRecomendado': recomendados.length,
        'mediaDificuldade': somaDificuldades / recomendados.length,
        'maiorScore': maiorScore,
        'titulos': titulos,
    };
}

void main() {
    final recomendados = recomendarConteudos(
        perfilUsuario: [0.9, 0.7, 0.2, 0.4, 0.8],
        conteudos: [
            ConteudoEstudo(
                id: 'c1',
                titulo: 'Vetores em IA',
                vetor: [1.0, 0.6, 0.1, 0.2, 1.0],
                dificuldade: 3,
                ativo: true,
                prioridade: 1.2,
            ),
            ConteudoEstudo(
                id: 'c2',
                titulo: 'OOP em Dart',
                vetor: [0.8, 1.0, 0.1, 0.2, 0.3],
                dificuldade: 2,
                ativo: true,
                prioridade: 1.1,
            ),
        ],
        dificuldadeMaxima: 3,
    );

    print(gerarResumoRecomendacao(recomendados));
}
```

Comentário: Aqui o Map funciona como um pequeno relatório. O tratamento da lista vazia impede erros ao acessar first e evita divisão por zero. Depois, a função percorre a lista uma vez para somar dificuldades, guardar títulos e descobrir o maior score.

Questão 4 - registrarRecomendacao em TypeScript

```
import { initializeApp } from 'firebase-admin/app';
import { getFirestore, FieldValue } from 'firebase-admin/firestore';
import { HttpsError, onCall } from 'firebase-functions/v2/https';

initializeApp();
const db = getFirestore();

type ConteudoFirestore = {
    titulo: string;
    vetor: number[];
    dificuldade: number;
    ativo: boolean;
    prioridade: number;
};

type ConteudoComScore = ConteudoFirestore & {
    id: string;
    scoreFinal: number;
};
```

```

};

function calcularSimilaridadeCosseno(a: number[], b: number[]): number {
  if (a.length !== b.length) {
    throw new HttpsError('invalid-argument', 'Vetores com tamanhos diferentes.');
```

```

  }

  if (a.length === 0) {
    throw new HttpsError('invalid-argument', 'O vetor não pode estar vazio.');
```

```

  }

  let produtoEscalar = 0;
  let somaQuadradosA = 0;
  let somaQuadradosB = 0;
```

```

  for (let i = 0; i < a.length; i++) {
    produtoEscalar += a[i] * b[i];
    somaQuadradosA += a[i] * a[i];
    somaQuadradosB += b[i] * b[i];
  }
```

```

  const moduloA = Math.sqrt(somaQuadradosA);
  const moduloB = Math.sqrt(somaQuadradosB);
```

```

  if (moduloA === 0 || moduloB === 0) {
    return 0;
  }
```

```

  return produtoEscalar / (moduloA * moduloB);
}
```

```

export const registrarRecomendacao = onCall(async (request) => {
  if (!request.auth) {
    throw new HttpsError('unauthenticated', 'Usuário não autenticado.');
```

```

  }

  const perfil = request.data?.perfil;
  const dificuldadeMaxima = request.data?.dificuldadeMaxima;
```

```

  if (!Array.isArray(perfil) || perfil.some((valor) => typeof valor !== 'number')) {
    throw new HttpsError('invalid-argument', 'Perfil inválido.');
```

```

  }

  if (typeof dificuldadeMaxima !== 'number') {
    throw new HttpsError('invalid-argument', 'Dificuldade máxima inválida.');
```

```

  }

  const snapshot = await db
    .collection('conteudos')
    .where('ativo', '==', true)
    .get();
```

```

  const candidatos: ConteudoComScore[] = [];
```

```

  snapshot.forEach((doc) => {
    const dados = doc.data() as ConteudoFirestore;

    if (dados.dificuldade > dificuldadeMaxima) {
      return;
    }
```

```

    const similaridade = calcularSimilaridadeCosseno(perfil, dados.vetor);
```

```

    candidatos.push({
      id: doc.id,
      ...dados,
      scoreFinal: similaridade * dados.prioridade,
    });
  });
```

```

  candidatos.sort((a, b) => {
    const diferencaScore = b.scoreFinal - a.scoreFinal;

    if (diferencaScore !== 0) {
      return diferencaScore;
    }
```

```

    return a.dificuldade - b.dificuldade;
  });
```

```

  const top3 = candidatos.slice(0, 3);
```

```

  await db
    .collection('usuarios')
    .doc(request.auth.uid)
    .collection('historicoRecomendacoes')
    .add({
      perfil,
      dificuldadeMaxima,
```

```
        recomendacoes: top3,  
        criadoEm: FieldValue.serverTimestamp(),  
    });  
  
    return {  
        total: top3.length,  
        recomendacoes: top3,  
    };  
});
```

Comentário: A função é onCall porque foi pensada para ser chamada pelo app Flutter via SDK. O request.auth é usado para validar o usuário. O Firestore fornece os conteúdos ativos, o backend calcula o score, ordena o ranking, salva o histórico dentro do usuário autenticado e retorna os três melhores itens. HttpsError é usado porque erros em onCall devem voltar ao cliente de forma padronizada.

Gabarito - Simulado 2

As funções foram separadas em blocos para facilitar a leitura. Em uma prova escrita, o importante é mostrar claramente a validação, a lógica de cálculo e o retorno esperado.

Questão 1 - calcularMediaMovel

```
double calcularMedia(List<double> valores, int inicio, int fim) {
    double soma = 0.0;

    for (int i = inicio; i <= fim; i++) {
        soma += valores[i];
    }

    return soma / (fim - inicio + 1);
}

List<double> calcularMediaMovel(List<double> valores, int janela) {
    if (janela <= 0) {
        throw ArgumentError('A janela precisa ser maior que zero.');
```

```
    }

    final List<double> medias = [];

    for (int i = 0; i < valores.length; i++) {
        int inicio = i - janela + 1;

        if (inicio < 0) {
            inicio = 0;
        }

        final media = calcularMedia(valores, inicio, i);
        medias.add(media);
    }

    return medias;
}

void main() {
    final valores = [10.0, 20.0, 30.0, 40.0];
    print(calcularMediaMovel(valores, 3));
}
```

Comentário: A função percorre cada posição da lista e calcula a média usando o valor atual e os valores anteriores que cabem na janela. No começo da lista, quando ainda não há elementos suficientes, a janela fica menor naturalmente. A validação impede uma janela zero ou negativa.

Questão 2 - LeituraSensor, AlertaSensor e detectarAnomalias

```
import 'dart:math';

class LeituraSensor {
    final String idSensor;
    final String tipo;
    final double valor;
    final DateTime instante;

    LeituraSensor({
        required this.idSensor,
        required this.tipo,
        required this.valor,
        required this.instante,
    });
}

class AlertaSensor {
    final LeituraSensor leitura;
    final double zScore;
    final String severidade;

    AlertaSensor({
        required this.leitura,
        required this.zScore,
        required this.severidade,
    });
}

double calcularMediaValores(List<LeituraSensor> leituras) {
    double soma = 0.0;

    for (final leitura in leituras) {
```

```

        soma += leitura.valor;
    }

    return soma / leituras.length;
}

double calcularDesvioPadrao(List<LeituraSensor> leituras, double media) {
    double somaQuadrados = 0.0;

    for (final leitura in leituras) {
        final diferenca = leitura.valor - media;
        somaQuadrados += diferenca * diferenca;
    }

    return sqrt(somaQuadrados / leituras.length);
}

String classificarSeveridade(double zScoreAbsoluto) {
    if (zScoreAbsoluto >= 3.0) {
        return 'critica';
    }

    return 'moderada';
}

List<AlertaSensor> detectarAnomalias(
    List<LeituraSensor> leituras,
    double limiteZ,
) {
    if (leituras.isEmpty()) {
        return [];
    }

    if (limiteZ <= 0) {
        throw ArgumentError('O limiteZ precisa ser maior que zero.');
```

```

    print('${alerta.leitura.tipo}: ${alerta.zScore} - ${alerta.severidade}');
  }
}

```

Comentário: A solução separa responsabilidades: uma classe representa a leitura, outra representa o alerta. As funções auxiliares calculam média, desvio e severidade. O detector usa `abs()` para considerar tanto valores muito acima quanto muito abaixo da média.

Questão 3 - gerarResumoAlertas

```

Map<String, dynamic> gerarResumoAlertas(List<AlertaSensor> alertas) {
  final Map<String, int> alertasPorTipo = {};
  double maiorZScoreAbsoluto = 0.0;
  int quantidadeCritica = 0;

  for (final alerta in alertas) {
    final tipo = alerta.leitura.tipo;

    alertasPorTipo[tipo] = (alertasPorTipo[tipo] ?? 0) + 1;

    final zAbsoluto = alerta.zScore.abs();

    if (zAbsoluto > maiorZScoreAbsoluto) {
      maiorZScoreAbsoluto = zAbsoluto;
    }

    if (alerta.severidade == 'critica') {
      quantidadeCritica++;
    }
  }

  return {
    'totalAlertas': alertas.length,
    'alertasPorTipo': alertasPorTipo,
    'maiorZScoreAbsoluto': maiorZScoreAbsoluto,
    'quantidadeCritica': quantidadeCritica,
  };
}

void main() {
  final leitura1 = LeituraSensor(
    idSensor: 'S1',
    tipo: 'temperatura',
    valor: 45.0,
    instante: DateTime.now(),
  );

  final leitura2 = LeituraSensor(
    idSensor: 'S2',
    tipo: 'ruído',
    valor: 95.0,
    instante: DateTime.now(),
  );

  final alertas = [
    AlertaSensor(leitura: leitura1, zScore: 2.4, severidade: 'moderada'),
    AlertaSensor(leitura: leitura2, zScore: 3.2, severidade: 'critica'),
  ];

  print(gerarResumoAlertas(alertas));
}

```

Comentário: O Map `alertasPorTipo` usa o padrão `(valorAtual ?? 0) + 1` para contar ocorrências. O restante da função calcula o maior z-score absoluto e a quantidade de alertas críticos em uma única passagem pela lista.

Questão 4 - receberLeituraSensor em TypeScript

```

import { initializeApp } from 'firebase-admin/app';
import { getFirestore, FieldValue, Timestamp } from 'firebase-admin/firestore';
import { onRequest } from 'firebase-functions/v2/https';

initializeApp();
const db = getFirestore();

type LeituraInput = {
  idSensor: string;
  tipo: string;
  valor: number;
};

```

```

function calcularMedia(valores: number[]): number {
  return valores.reduce((soma, valor) => soma + valor, 0) / valores.length;
}

function calcularDesvioPadrao(valores: number[], media: number): number {
  const somaQuadrados = valores.reduce((soma, valor) => {
    const diferenca = valor - media;
    return soma + diferenca * diferenca;
  }, 0);

  return Math.sqrt(somaQuadrados / valores.length);
}

function classificarSeveridade(zAbs: number): string {
  if (zAbs >= 3.0) {
    return 'critica';
  }

  if (zAbs >= 2.0) {
    return 'moderada';
  }

  return 'sem_alerta';
}

export const receberLeituraSensor = onRequest(async (req, res) => {
  if (req.method !== 'POST') {
    res.status(405).json({ erro: 'Use o método POST.' });
    return;
  }

  const dados = req.body as Partial<LeituraInput>;

  if (
    typeof dados.idSensor !== 'string' ||
    typeof dados.tipo !== 'string' ||
    typeof dados.valor !== 'number'
  ) {
    res.status(400).json({ erro: 'Dados inválidos.' });
    return;
  }

  const leituraRef = await db.collection('leituras').add({
    idSensor: dados.idSensor,
    tipo: dados.tipo,
    valor: dados.valor,
    criadoEm: FieldValue.serverTimestamp(),
  });

  const recentesSnapshot = await db
    .collection('leituras')
    .where('idSensor', '==', dados.idSensor)
    .orderBy('criadoEm', 'desc')
    .limit(20)
    .get();

  const valores = recentesSnapshot.docs
    .map((doc) => doc.data().valor)
    .filter((valor): valor is number => typeof valor === 'number');

  let zScore = 0;
  let alertaCriado = false;
  let severidade = 'sem_alerta';

  if (valores.length > 0) {
    const media = calcularMedia(valores);
    const desvio = calcularDesvioPadrao(valores, media);

    if (desvio !== 0) {
      zScore = (dados.valor - media) / desvio;
      const zAbs = Math.abs(zScore);
      severidade = classificarSeveridade(zAbs);

      if (zAbs >= 2.0) {
        await db.collection('alertas').add({
          idSensor: dados.idSensor,
          tipo: dados.tipo,
          valor: dados.valor,
          leituraId: leituraRef.id,
          zScore,
          severidade,
          criadoEm: Timestamp.now(),
        });

        alertaCriado = true;
      }
    }
  }
}

```

```
res.status(201).json({
  leituraId: leituraRef.id,
  zScore,
  severidade,
  alertaCriado,
});
});
```

Comentário: Essa função é `onRequest` porque se comporta como uma rota HTTP. Ela valida o método, valida o corpo da requisição, salva a leitura, consulta leituras recentes do mesmo sensor e calcula o z-score da nova entrada. Se o limite for atingido, cria um documento em alertas. Os status 400, 405 e 201 separam erro de entrada, método incorreto e criação bem-sucedida.